COMP3311 Week 9 Thursday Lecture

- Week 09 Thursday
- PostgreSQL Query Costs
- EXPLAIN Examples
- Exercise: RA Trees to RA Plan
- Transactions, Concurrency
- Transactions
- Example Transaction
- Exercise: Transaction Failures
- Transaction Concepts
- Transaction Concepts (cont)
- Transaction Consistency
- Transaction Consistency (cont)
- Transactions
- Example Transaction
- Example Transaction (cont)
- Example Transaction (cont)
- Example Transaction (cont)
- Example Transaction (cont)
- Exercise: Transaction Failures
- Transaction Concepts
- Transaction Concepts (cont)
- Transaction Consistency
- Transaction Consistency (cont)
- Transactions, Concurrency
- Transactions, Concurrency (cont)
- Transactions
- Exercise: RA Tree to RA Plan

◆ Week 09 Thursday

In today's lecture ...

- EXPLAIN, Transactions, Concurrency

Things to do ...

- Assignment 2 due by 23:59 next Tuesday 22 April
- Help Sessions ... Mon 12:5, Tue 14:5, Thu 10-12, Fri 10-12
- MyExperience ... please fill out, want ~40% response rate

Things to note ...

- Exam on Mon 12 May; two sessions: morning, afternoon

◆ PostgreSQL Query Costs

PostgreSQL provides the `explain` statement to

- give a representation of the query execution plan
- with information that may help to tune query performance

Usage:

```
EXPLAIN [ANALYZE] Query
```

Without **ANALYZE**, **EXPLAIN** shows plan with estimated costs.

With **ANALYZE**, **EXPLAIN** executes query and prints real costs.

Note that runtimes may show considerable variation due to buffering.

If simply want to know the runtime is ok, maybe `timing` is good enough

◆ EXPLAIN Examples

Note that PostgreSQL builds a query evaluation tree, rather than a linear plan, e.g.

```
select *
from   R
       join S on (R.a = S.a)
       join T on (T.a = S.a)
where  R.a > 1;

Tmp1 = R dot(a>1) S
Tmp2 = Tmp1 dot(a=a) T
Tmp3 = Select*(Tmp2)
Res  = Proj(Tmp3)
```

EXPLAIN effectively shows a pre-order traversal of the plan tree

◆ EXPLAIN Examples (cont)

Example: Select on indexed attribute

```
db=# explain select * from students where id=123475;
                          QUERY PLAN
-----------------------------------------------------------
 Index Scan using students_pkey on students
   (cost=0.00..8.27 rows=1 width=8)
   Index Cond: (id = 123475)

db=# explain analyze select * from students where id=123475;
                          QUERY PLAN
-----------------------------------------------------------
 Total runtime: 2.252 ms
 Index Scan using students_pkey on students
   (cost=0.00..8.30 rows=1 width=8)
   Index Cond: (id = 123475)
   Index Cond: (id = 123475)
 Planning Time: 0.058 ms
 Execution Time: 0.025 ms
```


◆ EXPLAIN Examples (cont)

Example: Select on non-indexed attribute

```
db=# explain select * from people where origin=1;
                          QUERY PLAN
-----------------------------------------------------------
 Seq Scan on people  (cost=0.00..19.178.35 rows=1323 width=37)
   Filter: (origin = 1)

db=# explain analyze select * from people where origin=1;
                          QUERY PLAN
-----------------------------------------------------------
 Seq Scan on people
   (cost=0.00..19.178.35 rows=1323 width=37)
   (actual time=0.015..0.173..0.157 rows=1323 loops=1)
   Filter: (origin = 1)
   Rows Removed by Filter: 7185
 Planning Time: 0.058 ms
 Execution Time: 1.199 ms
```


◆ EXPLAIN Examples (cont)

Example: aggregate on an indexed attribute

```
db=# explain analyze select count(*) from Peoples
                          QUERY PLAN
-----------------------------------------------------------
 Result  (cost=51.31..51.32 rows=1 width=4)
   (actual time=0.205..0.207 rows=1 loops=1)
 InitPlan 1 (returns $0)
   -> Limit  (cost=0.29..51.31 rows=1 width=4)
         (actual time=0.027..0.027 rows=1 loops=1)
         -> Index Only Scan Backward using people_pkey on people
               (cost=0.29..19.178.35 rows=1323 width=37)
               (actual time=0.016..0.016 rows=1 loops=1)
               Index Cond: (id IS NOT NULL)
 Planning Time: 0.080 ms
 Execution Time: 0.234 ms

db=# explain analyze select count(*) from Peoples;
                          QUERY PLAN
-----------------------------------------------------------
 Result  (cost=51.31..51.32 rows=1 width=4)
   (actual time=0.205..0.207 rows=1 loops=1)
 InitPlan 1 (returns $0)
   -> Limit  (cost=0.29..51.31 rows=1 width=4)
         (actual time=0.027..0.027 rows=1 loops=1)
         -> Index Only Scan Backward using people_pkey on people
               (cost=0.29..19.178.35 rows=1323 width=37)
               (actual time=0.016..0.016 rows=1 loops=1)
               Index Cond: (id IS NOT NULL)
 Planning Time: 0.080 ms
 Execution Time: 0.234 ms
```


◆ EXPLAIN Examples (cont)

Example: Join on a primary key (indexed) attribute

```
db=# explain analyze select s.id, p.id, p.full_name
db=# from students s join People p on s.id=p.id;
                          QUERY PLAN
-----------------------------------------------------------
 Hash Join  (cost=51.31..71.24 rows=1323 width=21)
            (actual time=1.016..1.190 rows=1 loops=1)
   Hash Cond: (p.id = s.id)
   -> Seq Scan on people p
         (cost=0.00..17.178 rows=8500 width=20)
         (actual time=0.00 s..0.510 rows=8500 loops=1)
   Hash  (cost=0.00..19.00 rows=1000 width=1)
         (actual time=0.062..1.062 rows=8500 loops=1)
         Buckets: 8192  Batches: 1  Memory Usage: 279kB
   -> Seq Scan on students s
         (cost=0.00..19.00 rows=1000 width=1)
         (actual time=0.006..0.064 rows=8500 loops=1)
 Planning Time: 0.178 ms
 Execution Time: 1.349 ms
```


◆ EXPLAIN Examples (cont)

Example: Join on a non-indexed attribute

```
db=# explain analyze
db=# select s1.code, s2.code
db=# from Subjects s1 join Subjects s2 on s1.course=s2.course;
                          QUERY PLAN
-----------------------------------------------------------
 Hash Join  (cost=51.24..1037.42 rows=78551 width=18)
            (actual time=1.185..193.492 rows=78551 loops=1)
   Hash Cond: (s2.course = s1.course)
   -> Seq Scan on subjects s2
         (cost=0.00..45.59 rows=559 width=13)
         (actual time=0.010..0.150 rows=559 loops=1)
   -> Hash  (cost=0.00..45.59 rows=559 width=13)
         (actual time=0.020..1.027 rows=559 loops=1)
         Buckets: 1024  Batches: 1  Memory Usage: 35kB
   -> Seq Scan on subjects s1
         (cost=0.00..45.59 rows=559 width=13)
         (actual t (more.0.0..0.057 rows=559 loops=1)
 Planning Time: 0.112 ms
 Execution Time: 188.423 ms
```


◆ Exercise: RA Tree to RA Plan

Re-write the above tree-based plans as sequences of RA ops

Example:

```
Index Scan using students_pkey on students
   (cost=0.28..8.30 rows=1 width=8)
   Index Cond: (id = 123475)
```

is simply

```
Res = Select(id=123475)(Students
```


◆ Transactions, Concurrency

DBMSs provide valuable information resources in an environment that is:

- **shared** - concurrent access by multiple users
- **unstable** - potential for hardware/software failure

Each user should see the system as:

- **unshared** - their work is not inadvertantly affected by others
- **stable** - the data survives in the face of system failures

Ultimate goal: data integrity is maintained at all times.

◆ Transactions, Concurrency (cont)

Transaction processing

- techniques for managing "logical units of work" which may require multiple DB operations

Concurrency control

- techniques for ensuring that multiple concurrent transactions do not interfere with each other

Recovery mechanisms

- techniques to restore information to a consistent state, even after major hardware shutdowns/failures

In COMP3311, we consider only transactions and concurrency

◆ Transactions

A transaction is

- an atomic "unit of work" in an application
- which may require multiple database changes

Transactions happen in a multi-user, unreliable environment.

To maintain integrity of data, transactions must be:

- Atomic - either fully completed or completely rolled-back
- Consistent - map DB between consistent states
- Isolated - transactions do not interfere with each other
- Durable - persistent, restorable after system failures

◆ Example Transaction

Bank funds transfer

- move N dollars from account X to account Y
- **Accounts(id,name,balance,holds, ...)**
- **Branches(id,name,address,assets, ...)**
- maintain **Branches.assets** as sum of balances via triggers
- transfer implemented by function which
  - has three parameters: amount, source acct, dest acct
  - checks validity of supplied accounts
  - checks sufficient available funds
  - returns a unique transaction ID on success

◆ Example Transaction (cont)

Example function to implement bank transfer ...

```
create or replace function
  transfer(amount integer, Src text, Dest text)
  returning integer
declare
  a20 integer; a20 integer; avail integer;
begin
  select id,balance into a20,avail
  from   Accounts where name=Src;
  if (a20 is null) then
    raise exception 'Invalid source account %',Src;
  end if;
  select id into a20
  from   Accounts where name=Dest;
  if (a20 is null) then
    raise exception 'Invalid dest account %',Dest;
  end if;
...
```


◆ Example Transaction (cont)

Example function to implement bank transfer (cont)...

```
...
  if (avail < N) then
    raise exception 'Insufficient funds in %',Src;
  end if;
  -- update Accounts set balance = balance-N
  -- where id = a20;
  -- update Accounts set balance = balance+N
  -- where id = a20;
  update Accounts set balance = balance-N
  where  id = a20;
  update Accounts set balance = balance+N
  where  id = a20;
  return nextval('t_id_seq');
end;
```


◆ Exercise: Transaction Failures

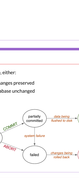
Describe any problems from the transaction failing after:

- after the first select
- after checking the validity of the accounts
- after checking sufficient balance
- after the first update
- after the second update

◆ Transaction Concepts

A transaction must always terminate, either:

- successfully (**COMMIT**), with all changes preserved
- unsuccessfully (**ABORT**), with database unchanged



◆ Transaction Concepts (cont)

To describe transaction effects, we consider:

- **READ** - transfer data from permanent storage (disk) to memory
- **WRITE** - transfer data from memory to permanent storage (disk)
- **ABORT** - terminate transaction, unsuccessfully
- **COMMIT** - terminate transaction, successfully

Normally abbreviated to **R(X); W(X); A; C**

- **SELECT** produces **READ**-operations on the database.
- **INSERT** produces **WRITE** operations.
- **UPDATE**, **DELETE** produce both **READ** + **WRITE** operations.

◆ Transaction Consistency

Transactions typically have intermediate states that are invalid.

However, states before and after transaction must be valid.

Database in consistent state
Transaction execution
Database in consistent state
consistent
inconsistent
consistent
consistent
consistent
consistent
consistent
inconsistent
consistent
consistent
consistent
consistent
cons

Arbitrary interleaving of operations causes anomalies, so that ...

- two consistency-preserving transactions
- produce a final state which is not consistent

◆ Serial Schedules

Serial execution: **T1** then **T2** or **T2** then **T1**

```
T1: R(X) W(X) R(Y) W(Y)
T2:                     R(X) W(X)
```

or

```
T1:                     R(X) W(X) R(Y) W(Y)
T2: R(X) W(X)
```

Serial execution guarantees a consistent final state if

- the initial state of the database is consistent
- **T1** and **T2** are consistency-preserving

◆ Concurrent Schedules

Concurrent schedules interleave **T1**,**T2**,... operations

Some concurrent schedules are ok, e.g.

```
T1: R(X) W(X)         R(Y)     W(Y)
T2:         R(X)         W(X)
```

Other concurrent schedules cause anomalies, e.g.

```
T1: R(X)      W(X)         R(Y) W(Y)
T2:     R(X)      W(X)
```

Want the system to ensure that only valid schedules occur.

◆ Serializability

Serializable schedule:

- concurrent schedule for $T_1..T_n$ with final state S
- S is also a final state of a serial schedule for $T_1..T_n$

Abstracting this needs a notion of schedule equivalence.

Two common formulations of serializability:

- conflict serializability (read/write operations occur in the "right" order)
- view serializability (read operations see the correct version of data)

◆ Conflict Serializability

A characterization of serializability based on conflicting operations

Consider two transactions T_1 and T_2 acting on data item X.

Possible orders for read/write operations by T_1 and T_2:

***T1* first**	***T2* first**	Equiv?
$R_1(X)\,R_2(X)$	$R_2(X)\,R_1(X)$	yes
$R_1(X)\,W_2(X)$	$W_2(X)\,R_1(X)$	no
$W_1(X)\,R_2(X)$	$R_2(X)\,W_1(X)$	no
$W_1(X)\,W_2(X)$	$W_2(X)\,W_1(X)$	no

Note: if T_1 and T_2 act on different data items (e.g. R(X), W(Y)), order does not matter.

◆ Conflict Serializability (cont)

Two transactions have a potential conflict if

- they perform operations on the same data item
- at least one of the operations is a write operation

In such cases, the order of operations affects the result.

If no conflict, can swap order without affecting the result.

If we can transform a schedule

- by swapping the order of non-conflicting operations
- such that the result is a serial schedule

then we say that the schedule is conflict serializible.

◆ Conflict Serializability (cont)

Example: transform a concurrent schedule to serial schedule

```
T1: R(A) W(A)         R(B)         W(B)
T2:         R(A)         W(A)         R(B) W(B)
swap
T1: R(A) W(A) R(B)                 W(B)
T2:         R(A)         W(A)         R(B) W(B)
swap
T1: R(A) W(A) R(B)         W(B)
T2:         R(A)         W(A) R(B) W(B)
swap
T1: R(A) W(A) R(B) W(B)
T2:         R(A)         W(A) R(B) W(B)
```


◆ Conflict Serializability (cont)

Checking for conflict-serializability:

- show that ordering in concurrent schedule
- cannot be achieved in any serial schedule

Method for doing this:

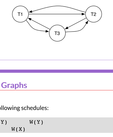
- build a precedence graph
- nodes represent transactions
- arcs represent order of action on shared data
- arc from $T_i \rightarrow T_j$ means T_i acts on X before T_j
- a cycle indicates schedule is *not* conflict-serializable.

◆ Example Precedence Graph

Example schedule which is not conflict serializable:

```
T1: R(X)         R(Y) W(X)         W(Y)
T2:         R(X)         W(X)
T3:         R(X)                   W(X)
```

Precendence graph for the above schedule:



◆ Exercise: Precedence Graphs

Build precedence graphs for the following schedules:

1.
```
T1: R(X) W(X)         R(Y)         W(Y)
T2:         R(X)         W(X)
```

2.
```
T1: R(X)         W(X)         R(Y) W(Y)
T2:         R(X)         W(X)
```

For each schedule:

- is it conflict serializable?
- if so, show the sequence of swaps that produces a serial schedule.

◆ View Serializability

View Serializability is

- an alternative formulation of serializability
- that is less conservative than conflict serializability (CS)
  (some safe schedules that are view serializable are not conflict serializable)

As with CS, it is based on a notion of schedule equivalence

- a schedule is "safe" if *view equivalent* to a serial schedule

The idea: if across the two schedules ...

- they read the same version of a shared object
- they write the same final version of an object

then they are view equivalent

◆ View Serializability (cont)

Two schedules S and S' on $T_1..T_n$ are view equivalent iff

- for each shared data item X
  - if, in S, T_i reads the initial value of X,
    then, in S', T_i also reads the initial value of X
  - if, in S, T_i reads X written by T_k
    then, in S', T_i also reads the value of X written by T_k
  - if, in S, T_i performs the final write of X,
    then, in S', T_i also performs the final write of X

To check serializibility of S ...

- find a serial schedule that is *view equivalent* to S
- from among the $n!$ possible serial schedules

◆ Exercise: Checking Serializability

Check whether each schedule is conflict/view serializable:

1.
```
T1: R(X)         W(X)
T2:         R(X)         W(X)
```

2.
```
T1: W(X)         R(Y)
T2:         R(Y)         R(X)
```

3.
```
T1: R(X)         W(X)
T2:         W(X)
T3:                      W(X)
```


◆ Concurrency Control

Serializability tests are useful theoretically ...

But don't provide a mechanism for organising schedules

- they can only be done "after the event"
- they are computationally very expensive $O(n!)$

What is required are methods that ...

- can be applied to each transaction individually
- guarantee that overall schedule is serializable

◆ Concurrency Control (cont)

Approaches to ensuring ACID transactions:

- lock-based
  Synchronise transaction execution via locks on some portion of the database.
- version-based
  Allow multiple consistent versions of the data to exist, and allow each transaction exclusive access to one version.
- timestamp-based
  Organise transaction execution in advance by assigning timestamps to operations.
- validation-based (optimistic concurrency control)
  Exploit typical execution-sequence properties of transactions to determine safety dynamically

◆ Lock-based Concurrency Control

Synchronise access to shared data items via following rules:

- before reading X get shared (read) lock on X
- before writing X get exclusive (write) lock on X
- an attempt to get a shared lock on X is blocked
  if another transaction already has exclusive lock on X
- an attempt to get an exclusive lock on X is blocked
  if another transaction has any kind of lock on X

These rules alone do not guarantee serializability ...

- but two-phase locking does
- acquire all needed locks before performing any unlocks

Locking also introduces potential for deadlock and starvation.

◆ Lock-based Concurrency Control (cont)

Examples of locking:

```
schedule 1
T1: L_s(X)         R(X)                 W(X) U(X)
T2:         L_s(Y)         R(Y) W(Y)         U(Y)

schedule 2
T1: L_s(X)         R(X) W(X) U(X)
T2:         L_s(X) ............ R(X) W(X) U(X)
```

New operations: **L_s()** = exclusive lock, **L_s()** = shared lock, **U()** unlock

Note: in Schedule 2, locking effectively forces serial execution

This is not generally the case; there may be some concurrency

In general, locking reduces concurrency, but gains safety